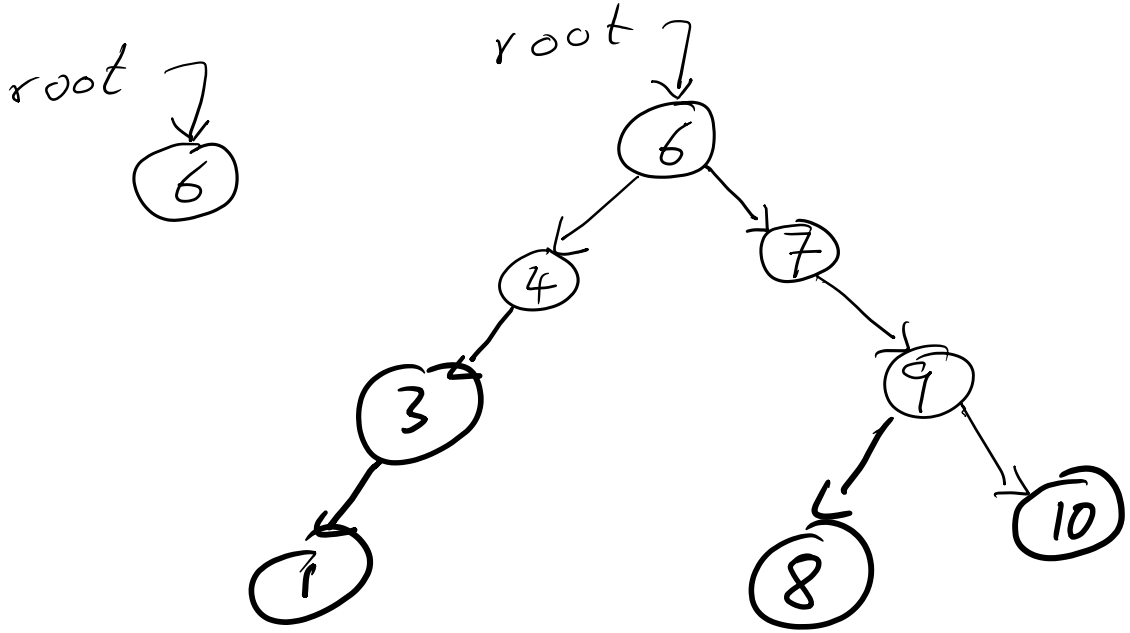
Trees and Rotations

6, 7, 4, 9, 10, 3, 1, 8

Insert into a BST, one-by-one.

root.

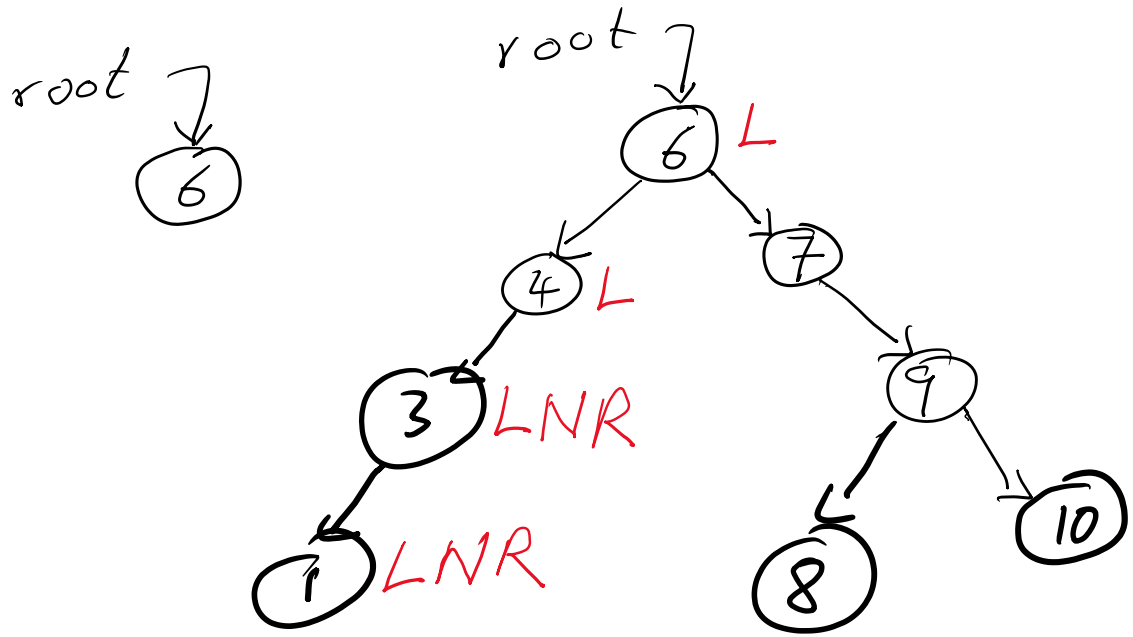


6, 7, 4, 9, 10, 3, 1, 8

Inorder, LNR

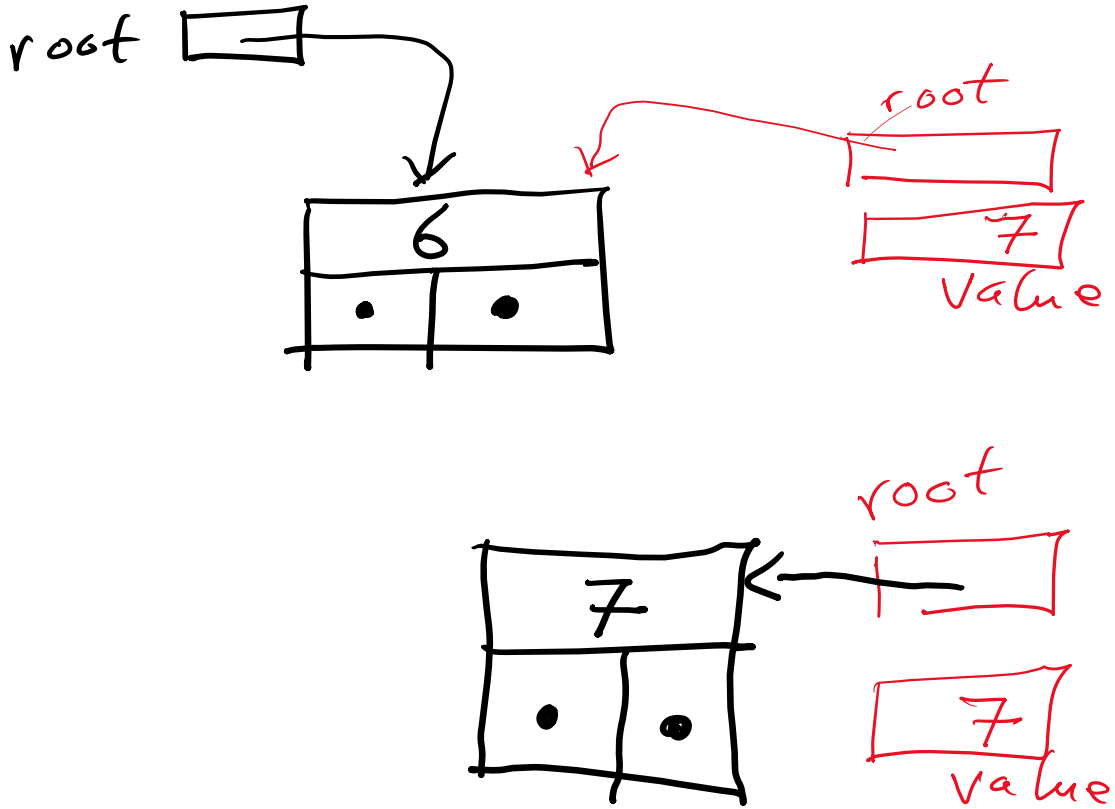
Insert into a BST, one-by-one.

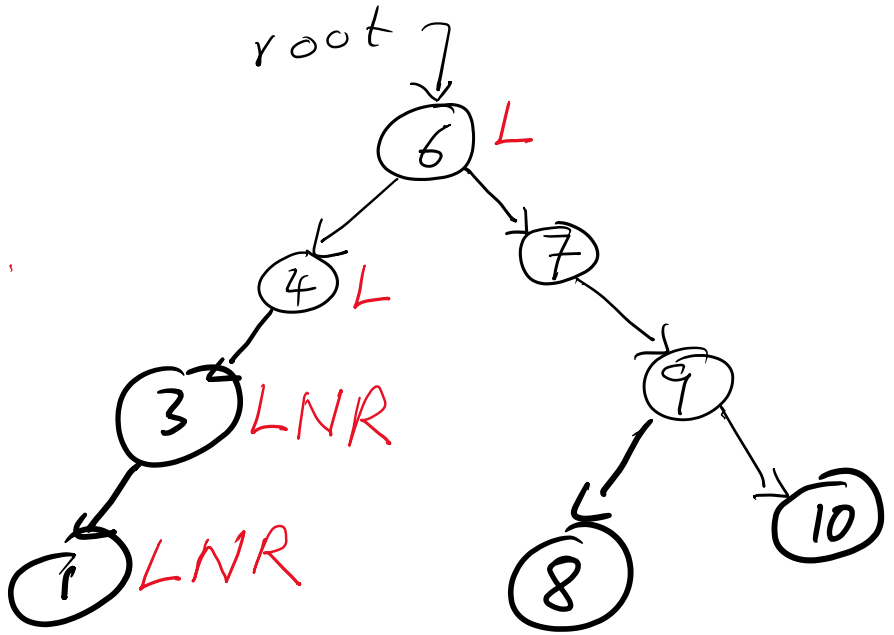
root.



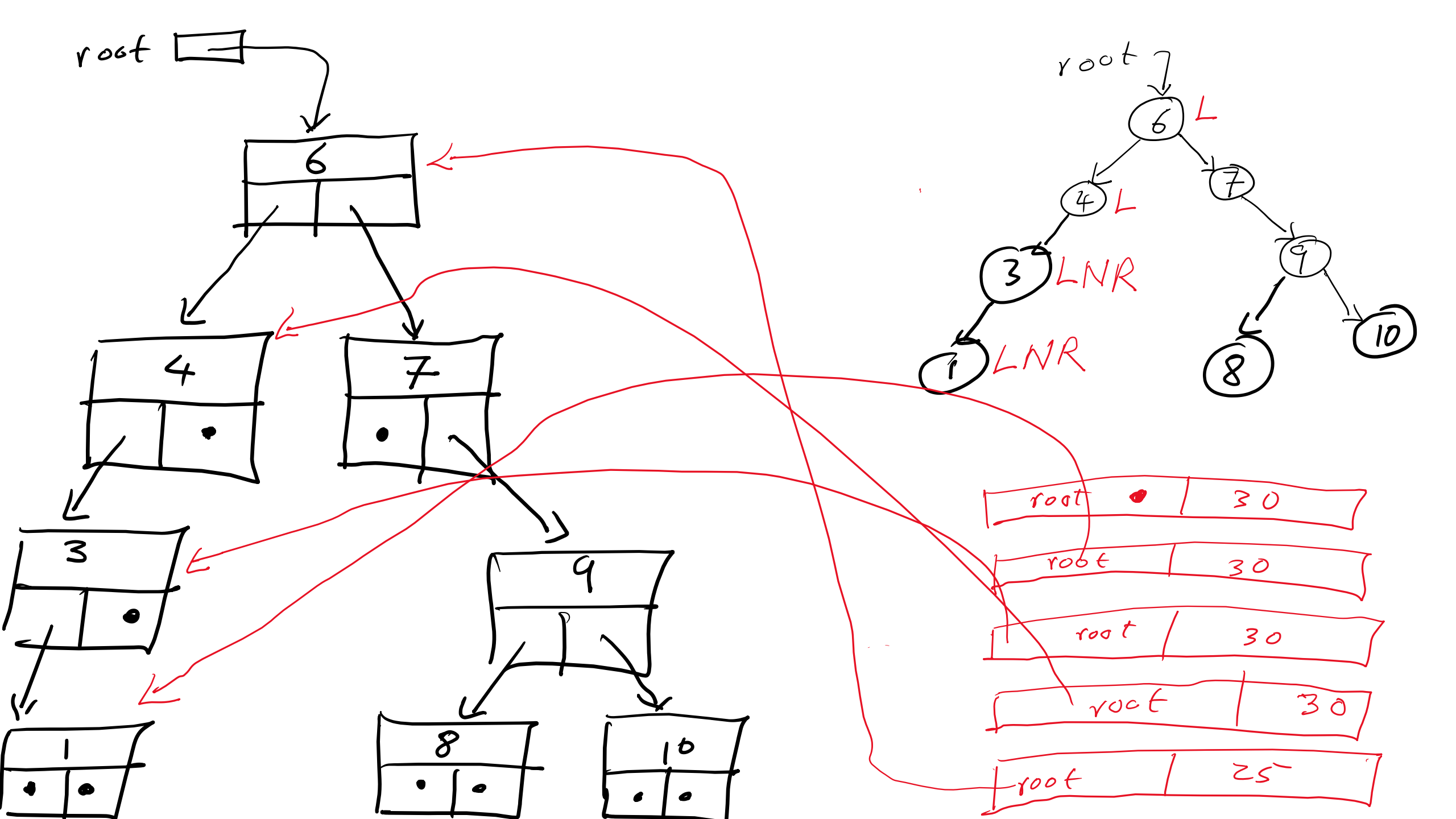
Output:

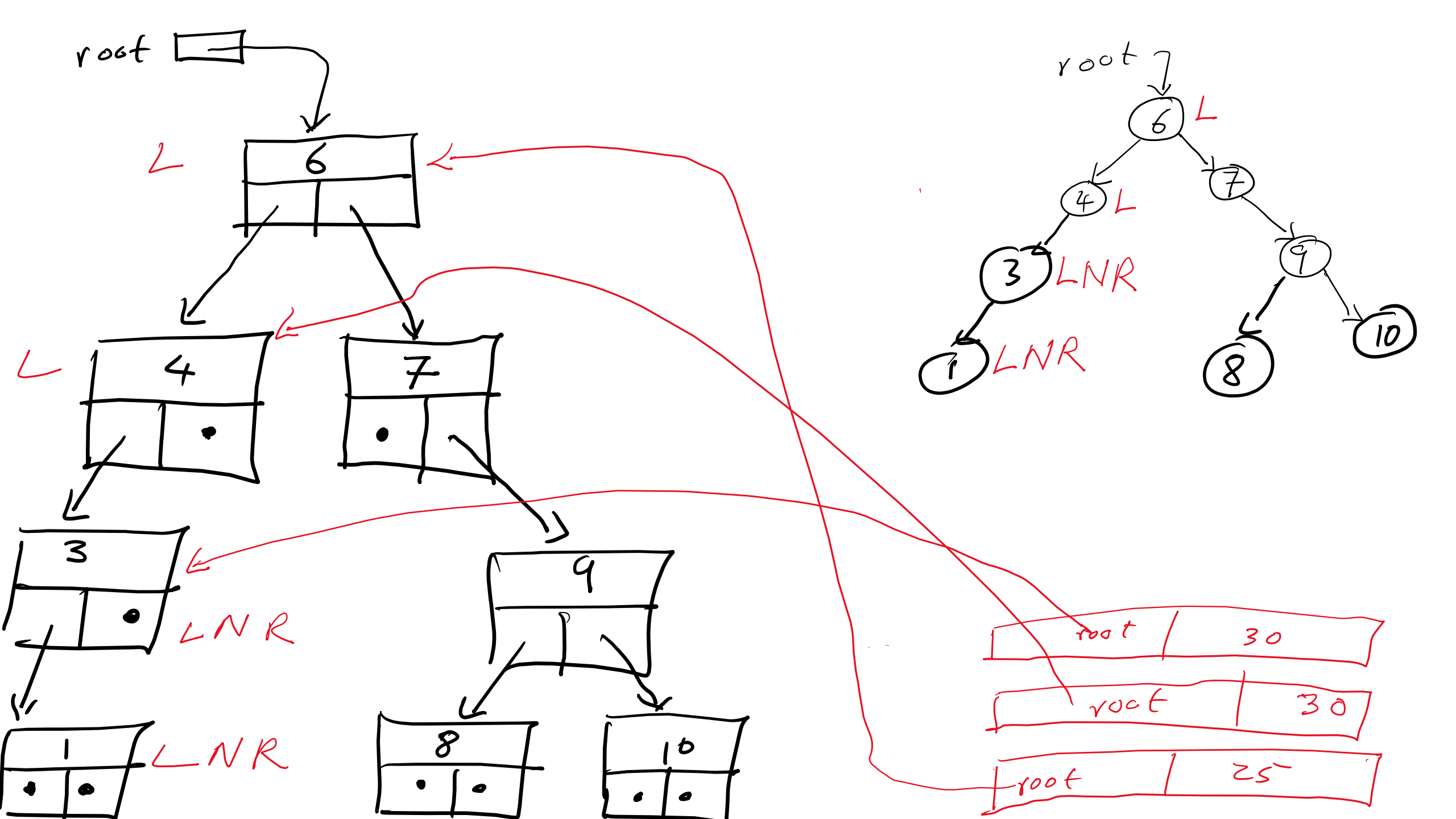
1 3


root
6
root
7
value
root
7
root
7
value


root
6
L
4
L
7
3
LNR
9
1
LNR
8
10


root.setRight(7, 19)
root
6
7


root
6
4
7
3
9
1
8
10
root
6
L
4
L
7
3
LNR
9
1
LNR
8
10
root
30
root
30
root
30
root
30
root
25


root
6
L
4
7
3
9
1
8
10
LNR
root
30
root
30
root
25

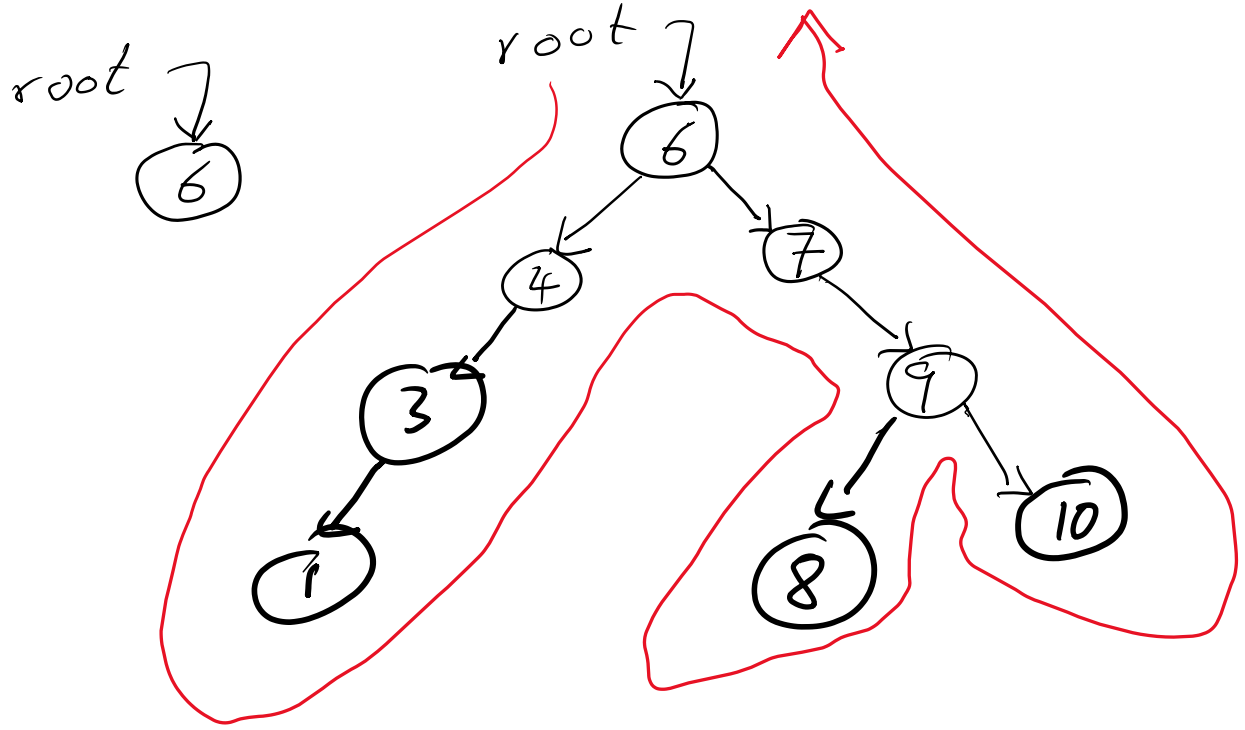
6, 7, 4, 9, 10, 3, 1, 8

Preorder

NLR

Insert into a BST, one-by-one.

root.



Output:

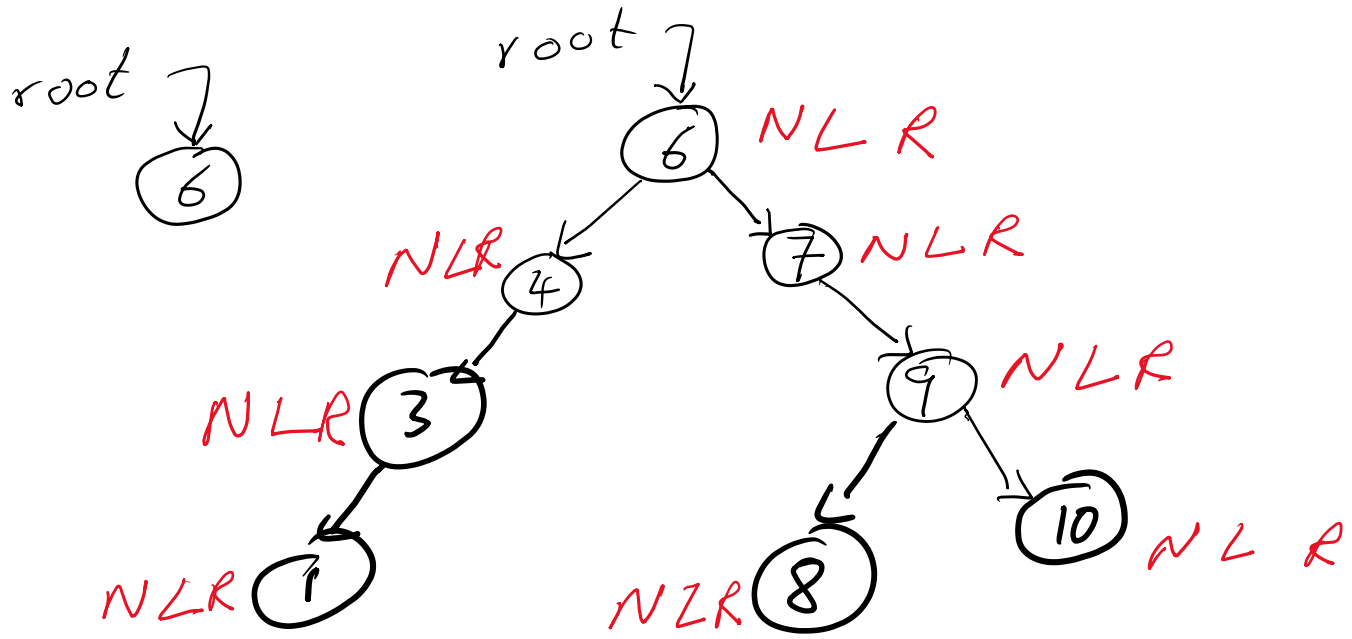
6 4 3 1 7 9 8 10

6, 7, 4, 9, 10, 3, 1, 8

NLR

Insert into a BST, one-by-one.

root.



Output

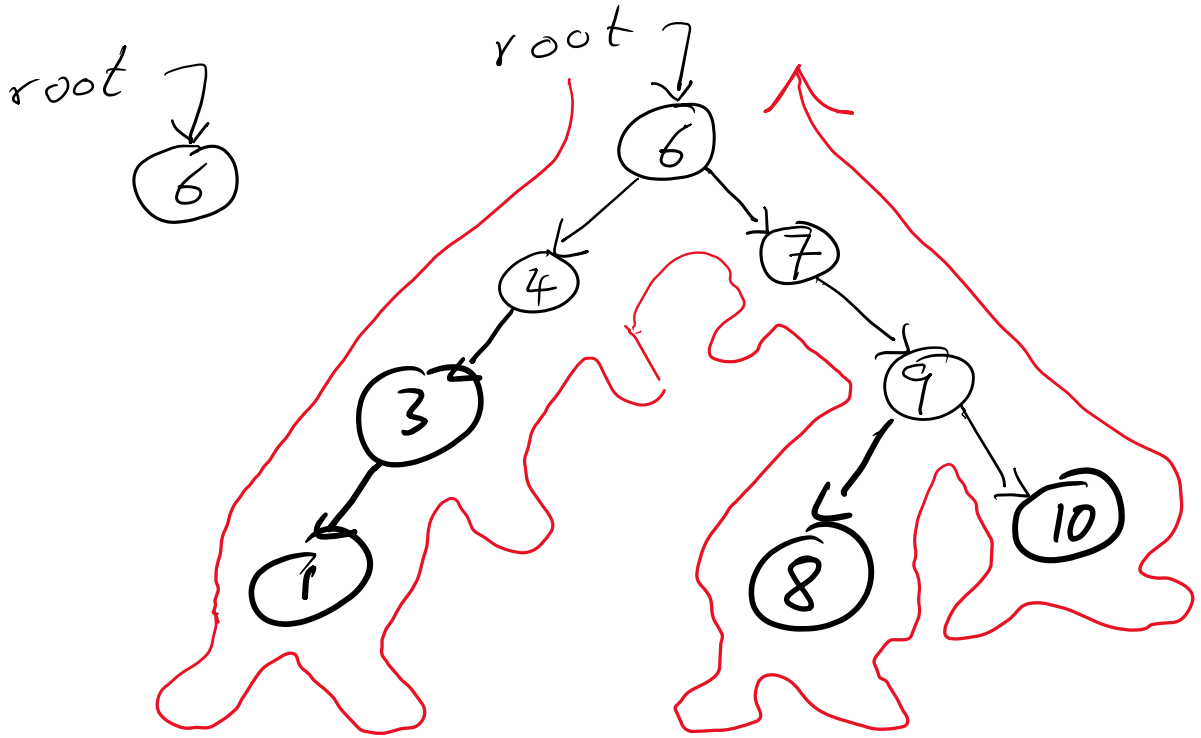
6 4 3 1 7 9 8 10

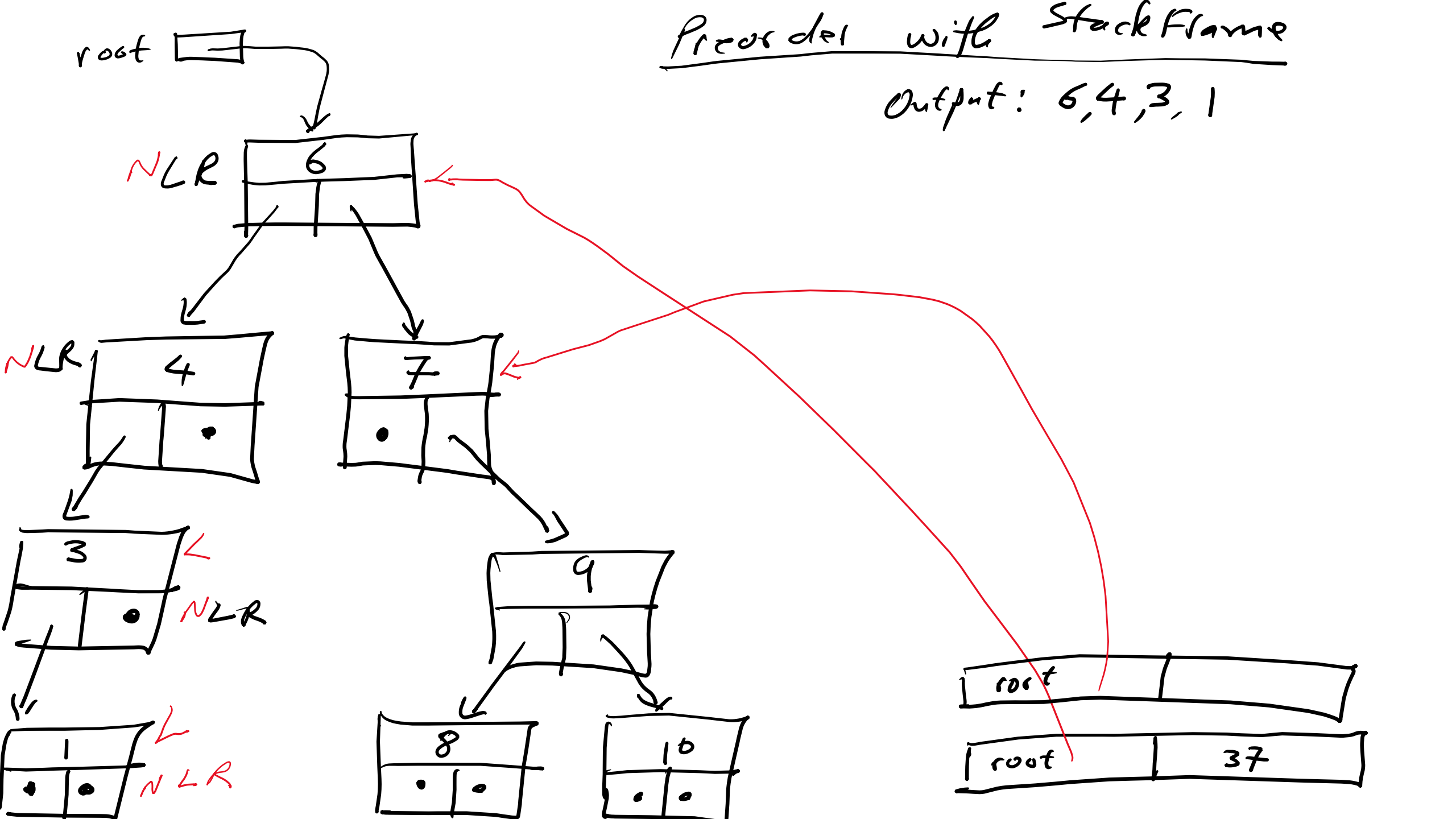
6, 7, 4, 9, 10, 3, 1, 8

Inorder

Insert into a BST, one-by-one.

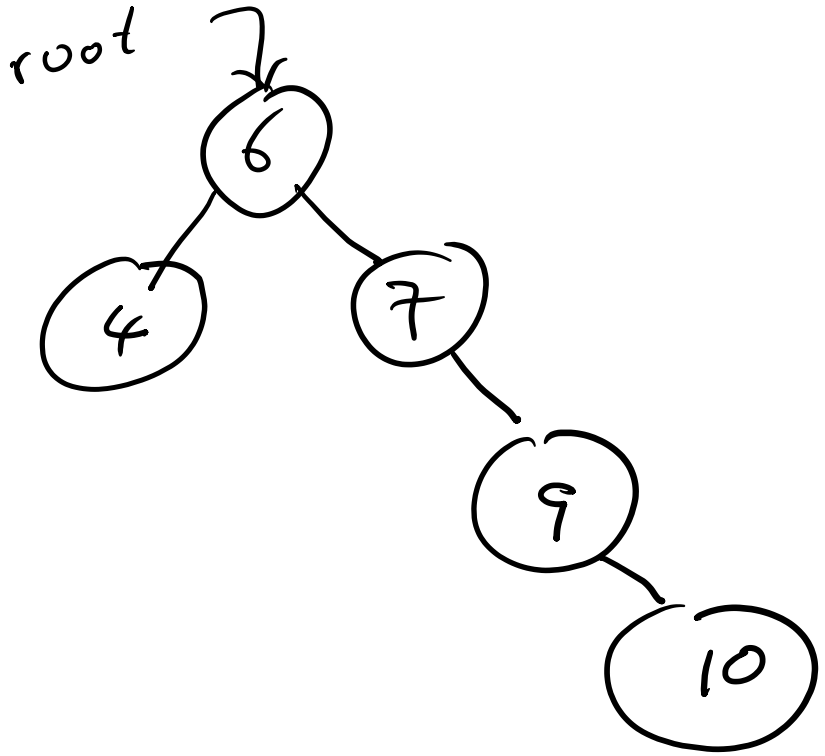
root.



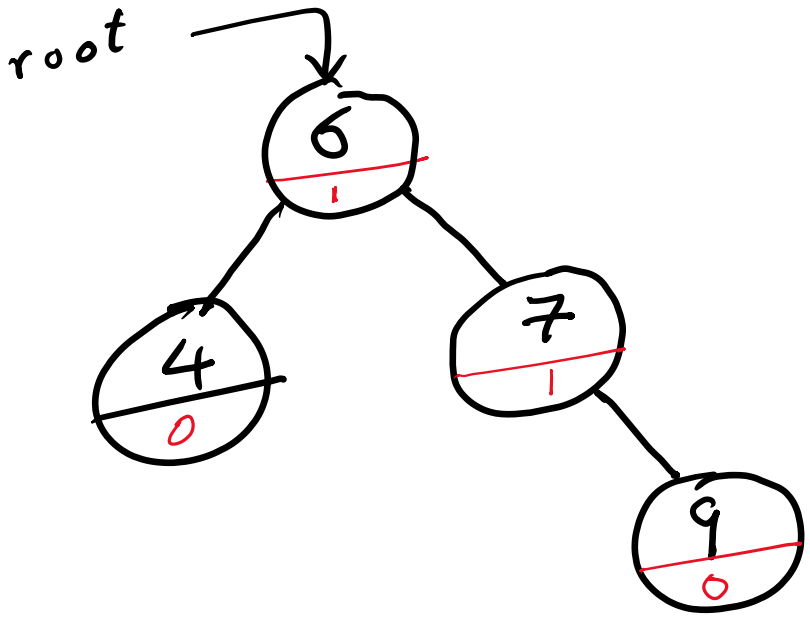

Preorder with StackFrame
Output: 6,4,3,1
root
NLR
6
NLR
4
7
3
NLR
9
1
NLR
8
10
root
root
37

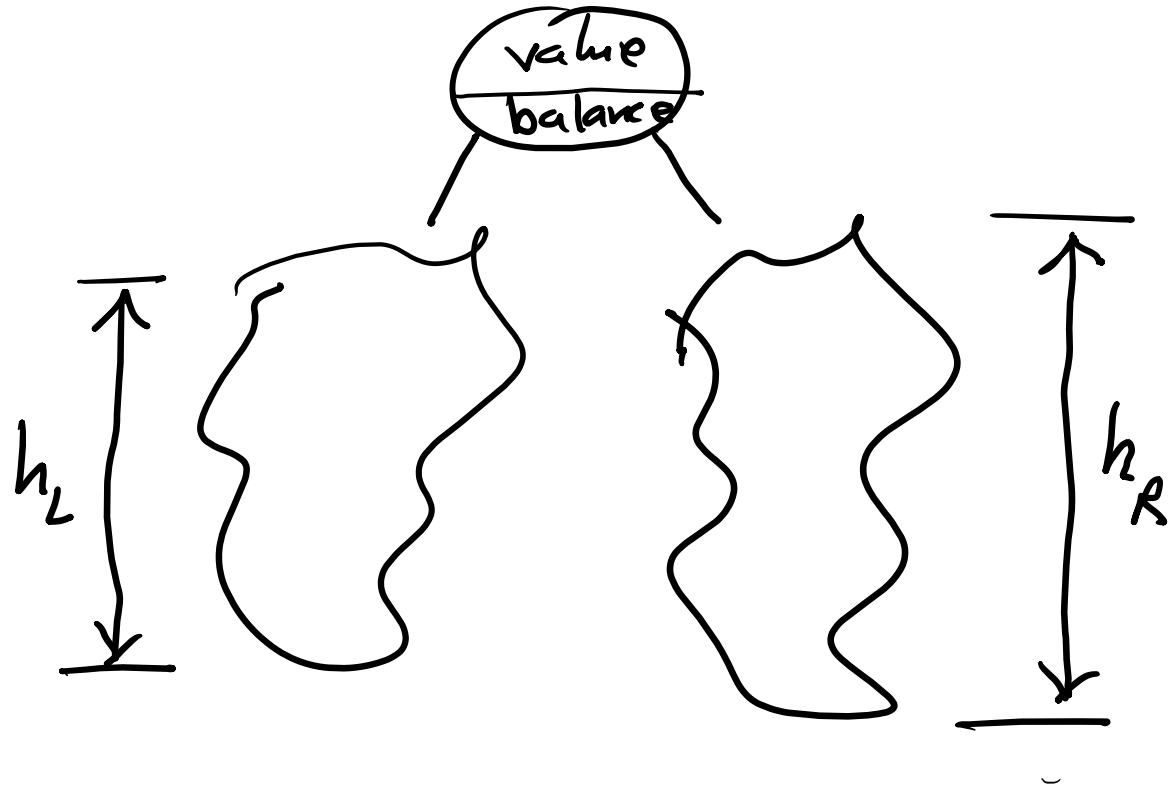
- A tree which is not balanced is inefficient to search. A degenerate tree where all items are inserted in ascending or descending order has the appearance and efficiency of a list (O(n)). A balanced tree is searchable in O(log n) time.

6, 7, 4, 9, 10, 3, 1, 8



6, 7, 4, 9, 10, 3, 1, 8





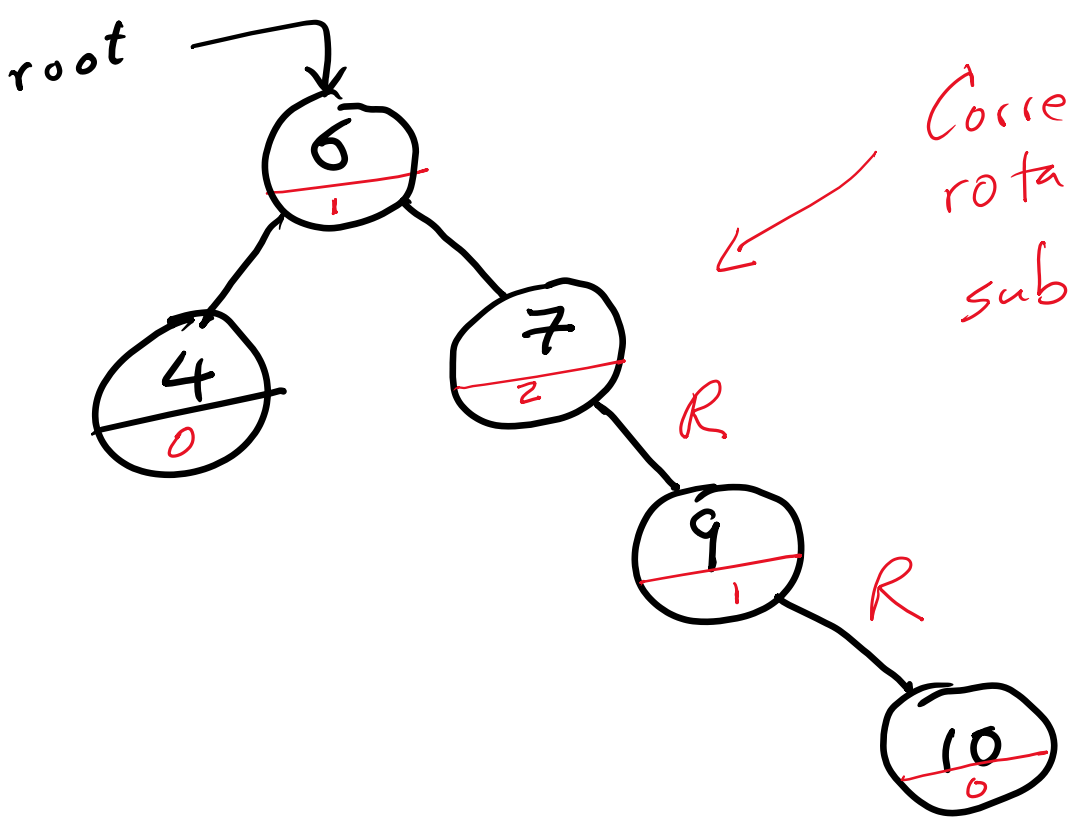
Good balances are calculated $h_R - h_L$

Values are: 0, -1 and +1

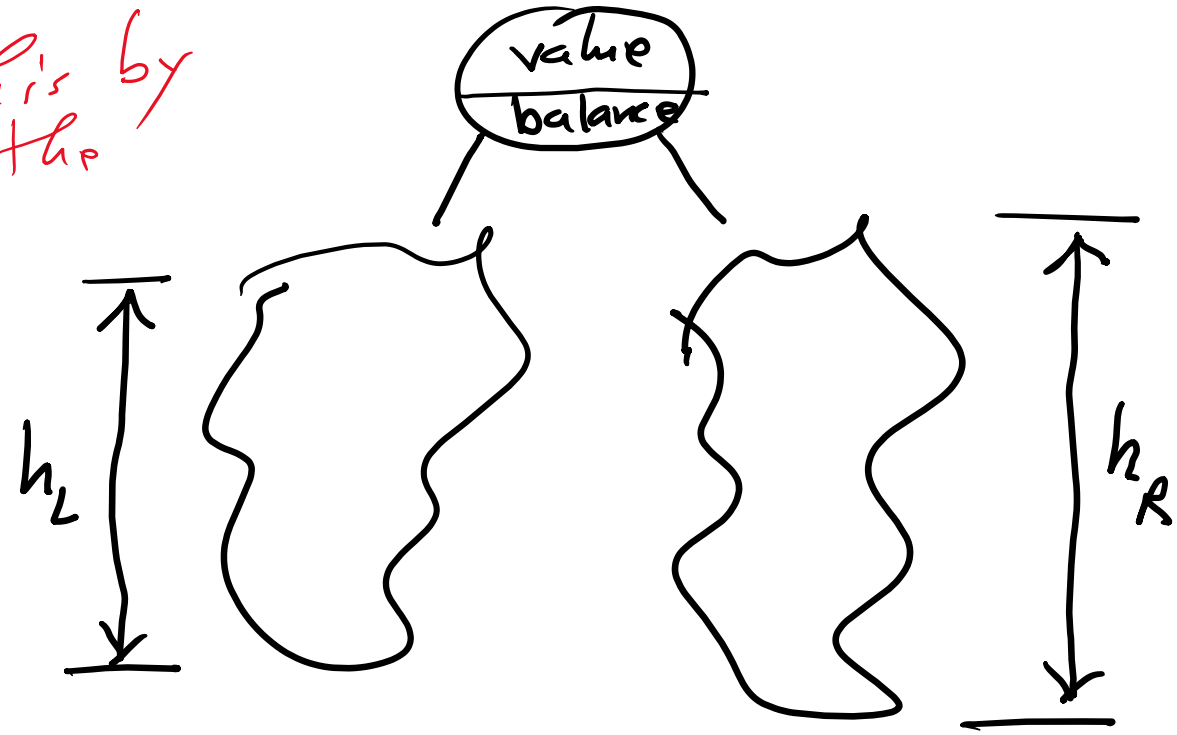
h_L height of left subtree

h_R height of right subtree

6, 7, 4, 9, 10, 3, 1, 8



Correct this by rotating the sub-tree



Good balances are calculated $h_R - h_L$

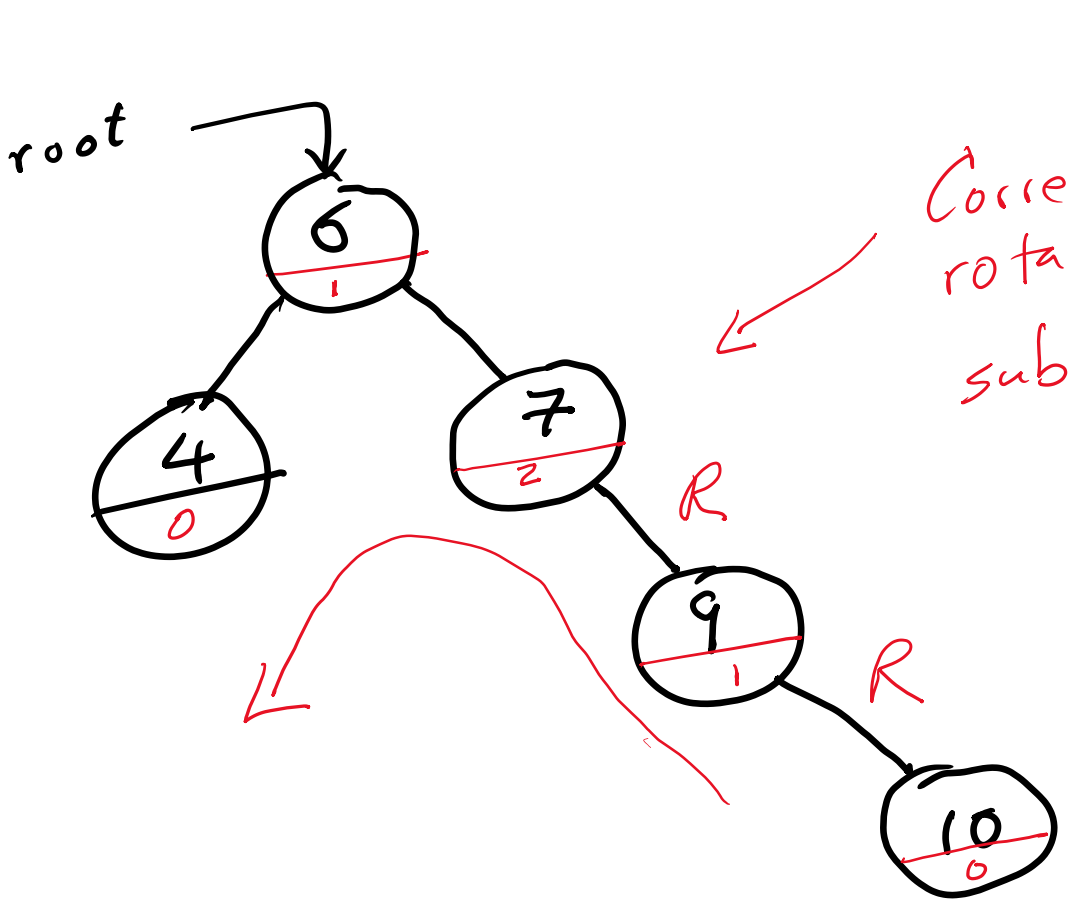
Values are: 0, -1 and +1

h_L height of Left subtree

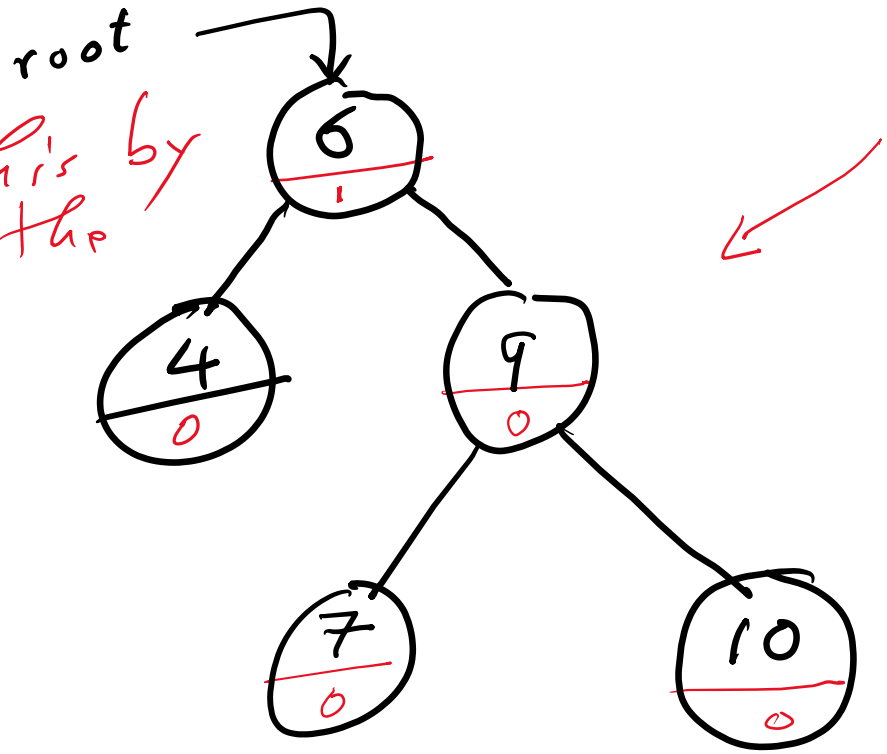
h_R height of right subtree

- In this case of imbalance, the path from the point of imbalance to the new node starts with going Right, and Right Again. Usually shortened to RR.
- To correct this situation for an AVL tree we need to perform a RR-Single Rotation operation.

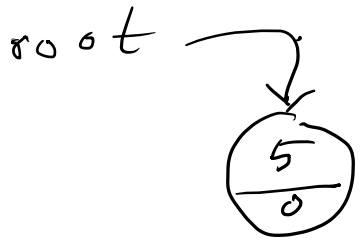
RR - Single Rotation

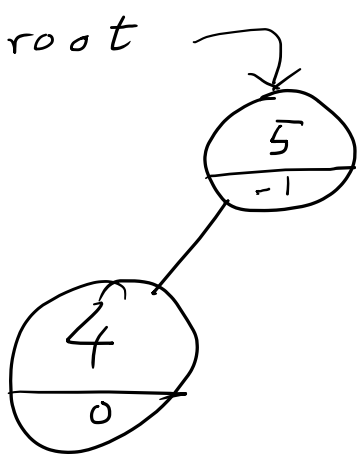


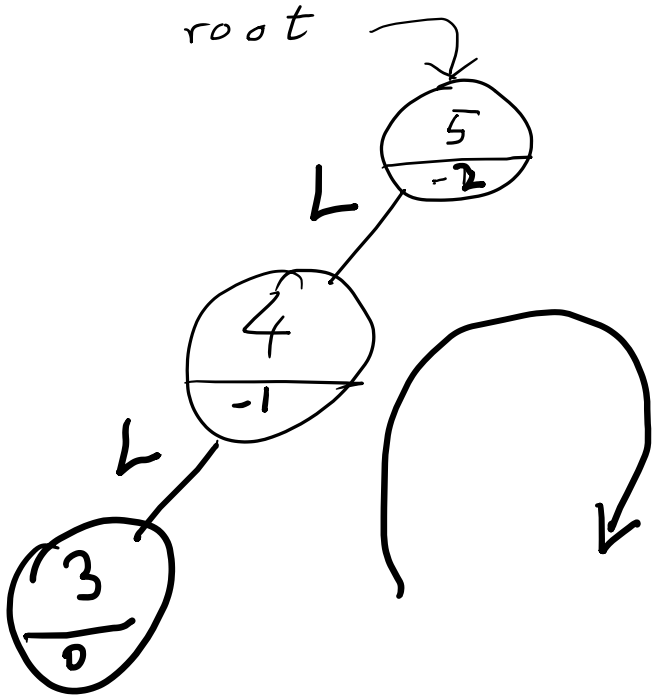
Correct this by rotating the sub-tree



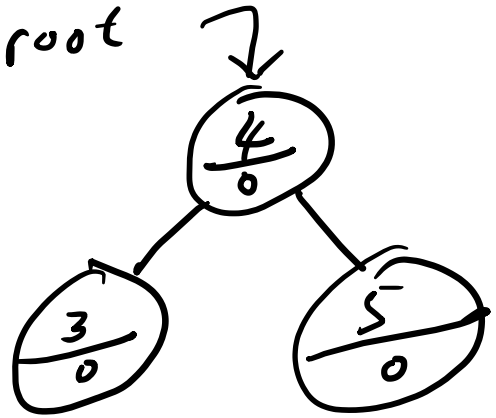
Correct rotating sub-tre





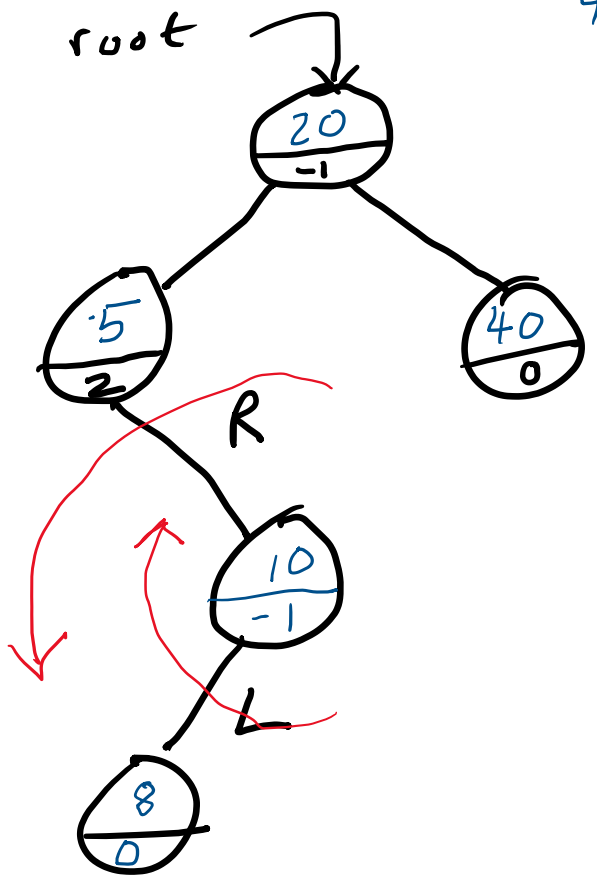


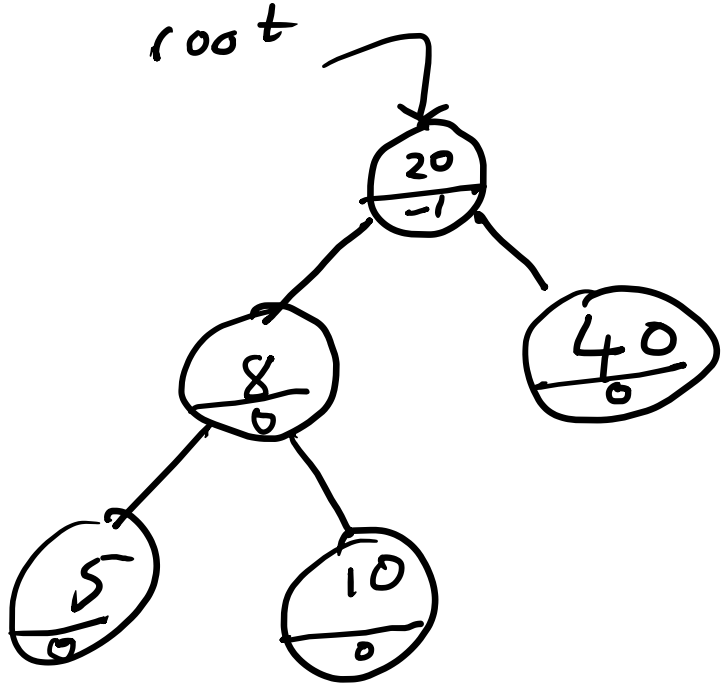
LL single Rotation



Insert 20, 40, 5, 10, 8

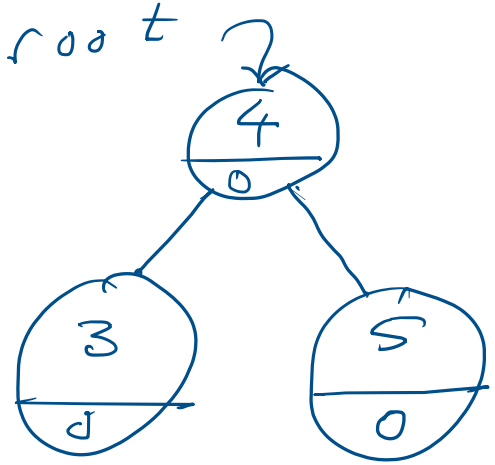
R-L Double Rotation

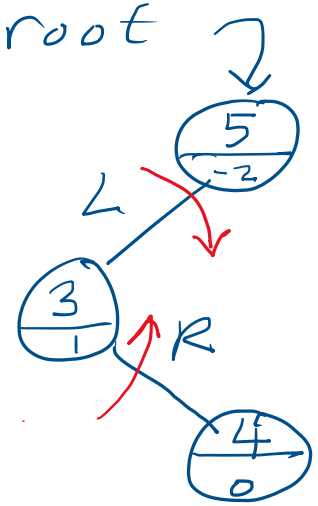




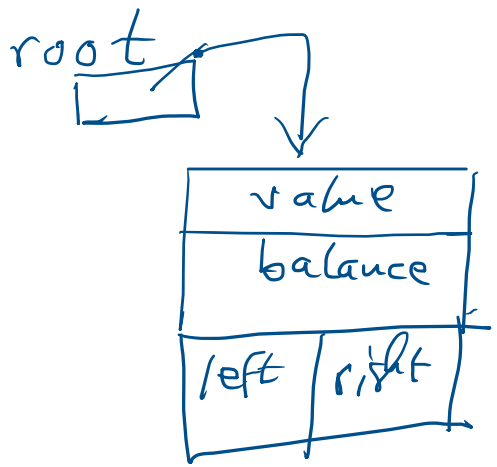
The path from the unbalanced node to the new node starts with Right the Left (RL)

LR Double Rotation





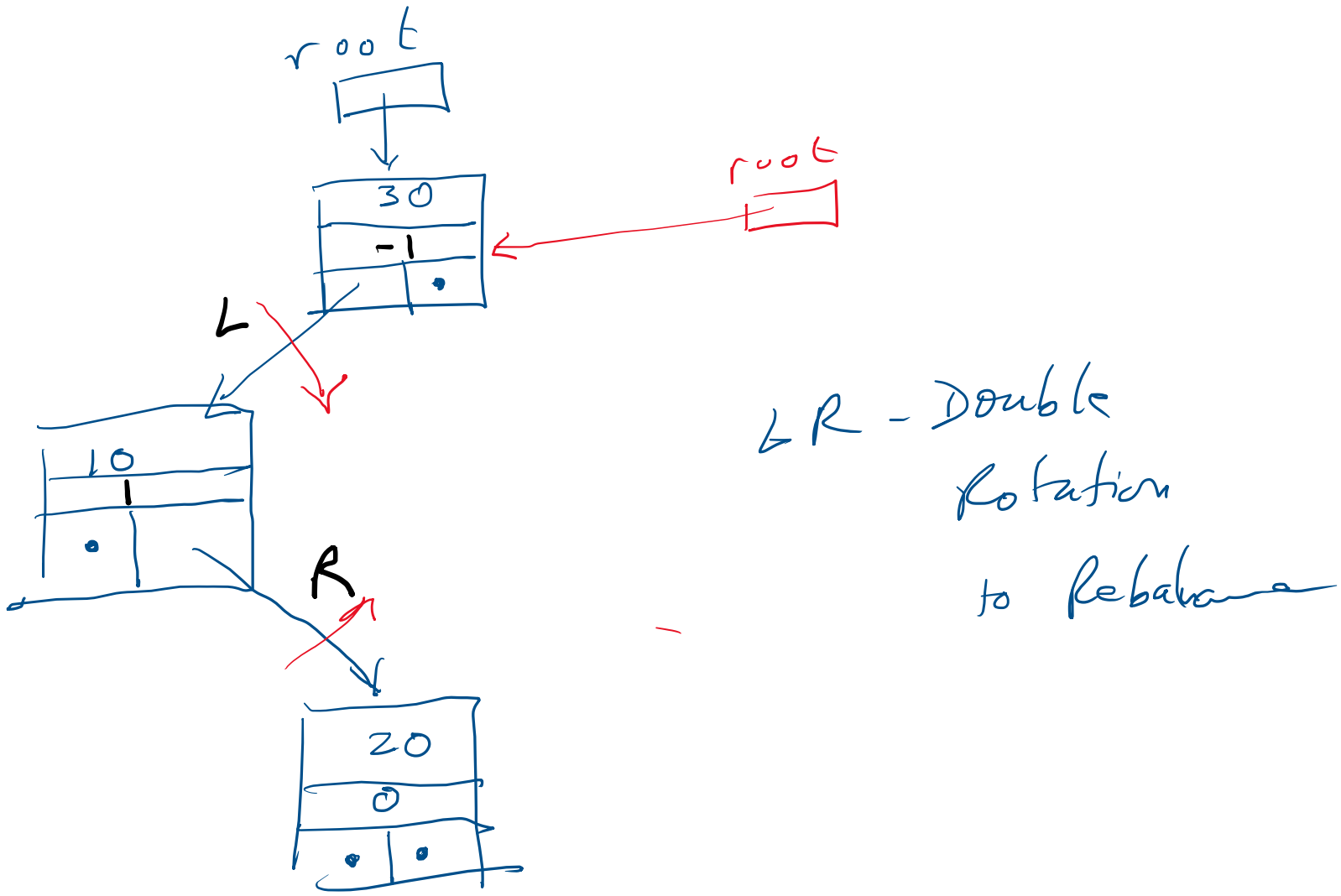
30, 10, 20, 40, 50, 25, 5, 3, 4, 35, 45, and 2

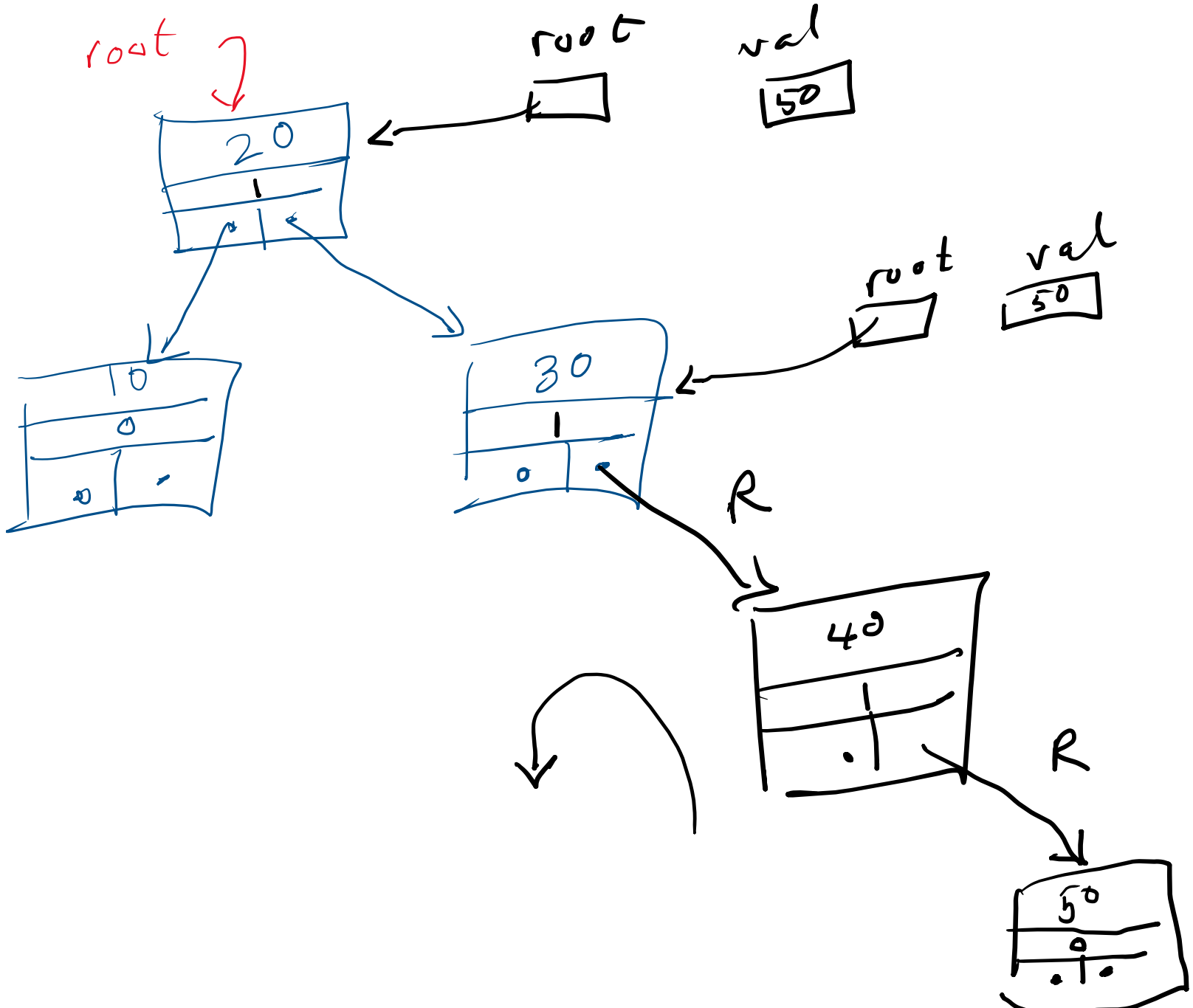


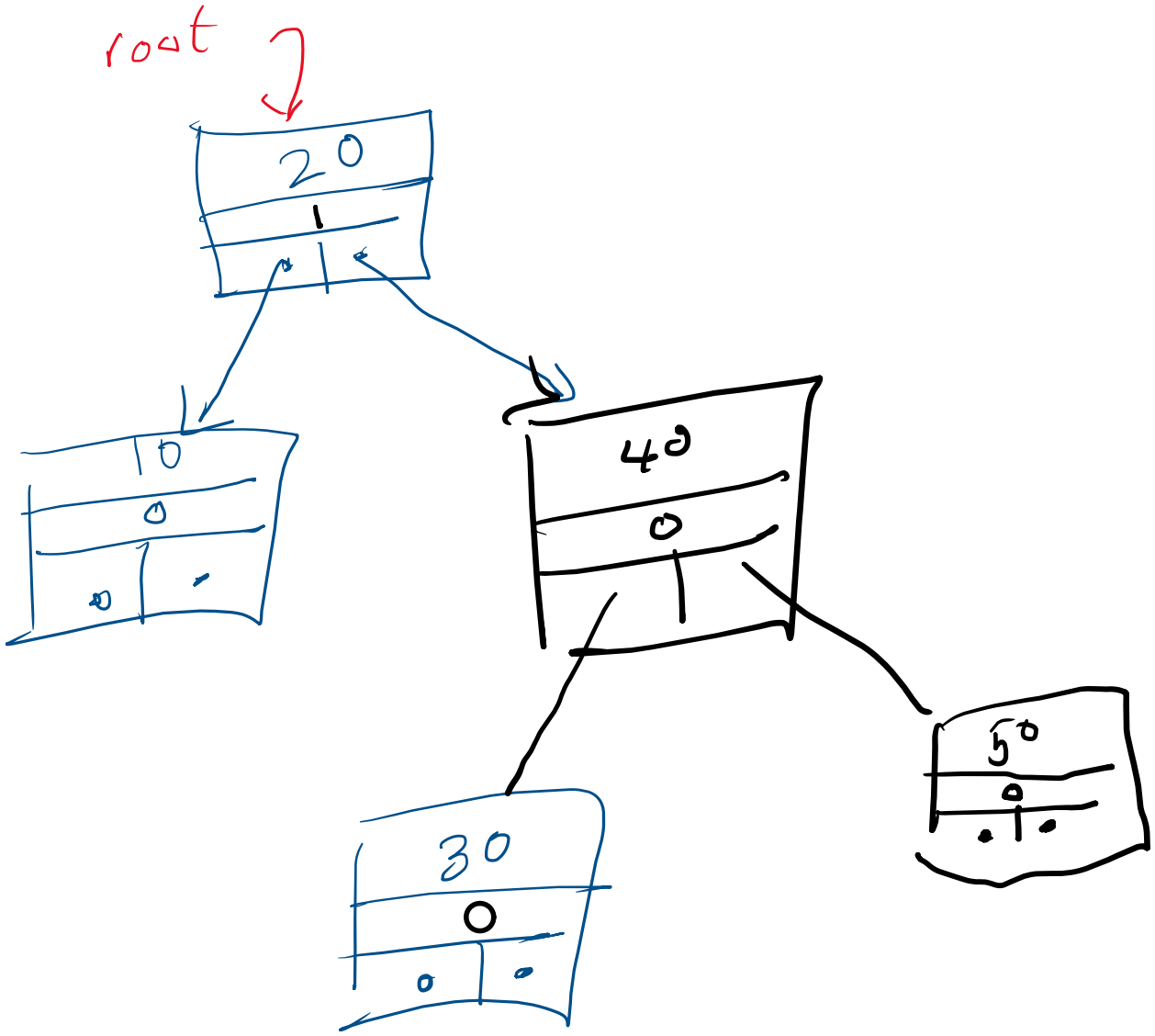

```
TreeNode root;

root = null;
```

30, 10, 20, 40, 50, 25, 5, 3,
4, 35, 45, and 2




root
root
val
50
20
1
10
0
30
1
root
val
50
R
40
R
50


root
20
10
40
30
50